COMPUTING INTEGRAL LENGTH OF SUMS OF SQUARES AND PYTHAGORAS ELEMENTS IN REAL QUADRATIC RINGS

TOMASZ KANIA

Abstract. `quadratic_sos` is a self-contained pure-Python reference package for exact integral sum-of-squares computation in maximal real quadratic orders. Given a squarefree radicand and a non-zero algebraic integer, the software decides whether the element is a sum of integral squares, computes its exact minimal length in the set of possible values one through five, or reports non-representability, and returns an explicit minimal decomposition when one exists. The package implements Peters' representability criterion, exact row-wise enumeration of relevant squares, meet-in-the-middle length detection, unit-square balancing along Pell-unit orbits, Pythagoras-number witnesses, and reproducible bounded-trace experiments. The accompanying article proves the exactness of these routines, explains the integer-only sign and search procedures, and analyses the balancing and finite-search phases. The repository contains an installable package, non-interactive drivers, regression tests, a brute-force validation sweep, scripts that regenerate the computational tables, an executed notebook, and performance plots. The implementation is intended as a readable executable specification and as a baseline for extensions to weighted diagonal representability.

ACM CCS 2012. Mathematics of computing – Mathematical software; Mathematics of computing – Algebraic algorithms; Mathematics of computing – Number theory; Computing methodologies – Symbolic and algebraic manipulation; Software and its engineering – Software libraries and repositories; Theory of computation – Design and analysis of algorithms.

1. Introduction

The repository `quadratic_sos`, maintained at https://github.com/tomaszkania/quadratic_sos, is the software artefact accompanying this article. The package exposes a compact exact API for real quadratic orders, representability tests, exact lengths, decomposition witnesses, and Pythagoras-number witnesses. It is written in pure Python to serve as an executable specification: the code is transparent, portable, and suitable for direct use in SageMath-style workflows or for translation into compiled arithmetic kernels. Regression tests, non-interactive drivers, validation sweeps, an executed notebook, and data-generation scripts are shipped with the repository. A companion repository, `quadratic_diagonal` [8], extends the same exact-search philosophy to arbitrary weighted diagonal lattices.

Let R be a commutative ring. Determining whether a given element of R is a sum of squares, and if so how many squares are necessary, is one of the oldest representation problems in arithmetic. Over global fields there are now explicit algorithms based on local–global principles and computable local invariants. Darkey-Mensah and Rothkegel computed lengths

2020 *Mathematics Subject Classification.* 11E25, 11R11, 11Y16, 68W30.

Key words and phrases. algorithms, sums of squares, real quadratic fields, rings of integers, length, Pythagoras number.

IM CAS (RVO 67985840).

and Pythagoras elements over arbitrary global fields [3]; Darkey-Mensah adapted the quadratic-form toolkit to global function fields of odd characteristic [2]; Koprowski later gave an explicit algorithm producing a decomposition of minimal length over every global field [9]. Recent companion work treats isotropic vectors over global fields and the computation of dyadic local square-class groups [11, 10]. Outside the global-field line, Kowalczyk and Miska studied Waring numbers of henselian rings [13], and Kowalczyk analysed sums of squares on rational surfaces [12]. In the integral setting, however, local–global principles fail in small rank, and the algorithmic problem changes substantially.

Real quadratic rings form the first natural test case where the arithmetic is both rich and still tractable. Let

$$K_D = \mathbb{Q}(\sqrt{D}), \qquad \mathcal{O}_D = \mathcal{O}_{K_D},$$

with $D \geqslant 2$ squarefree. For $0 \neq \alpha \in \mathcal{O}_D$ we define

$$\ell_D(\alpha) = \min\{n \in \mathbb{N} : \alpha = x_1^2 + \cdots + x_n^2,\ x_i \in \mathcal{O}_D\},$$

and set $\ell_D(\alpha) = \infty$ if no such representation exists. Our aim is the *element-wise exact-length problem*:

Given D and a specific element $\alpha \in \mathcal{O}_D$, compute the exact value of $\ell_D(\alpha)$.

Several structural results already exist in the literature. Peters characterised those elements of real quadratic orders that are sums of squares and proved an upper bound of 5 for the corresponding Pythagoras number [17, 18]. For maximal real quadratic orders, the exact values of the Pythagoras number are compiled in modern form by Krásenský, Raška and Sgallová [14, Theorem 3.1]; the family-wise problem of deciding when all elements of $m\mathcal{O}_D^+$ are sums of squares was studied by Kala and Yatsyna and then algorithmised by Raška [7, 19]. These papers are structural rather than element-wise algorithmic.

The novelty of the present paper is algorithmic rather than structural. We do not claim new arithmetic input; instead, we isolate a problem that had not previously been written down as an explicit algorithm in the integral real-quadratic setting and solve it in a form suitable both for proof and for exact implementation. More precisely, the present paper contributes:

(N1) the first explicit *element-wise exact-length algorithm* for maximal real quadratic orders, distinguishing the values $1, 2, 3, 4, 5, \infty$ for a given input element of $\mathcal{O}_D$;

(N2) a constant-cost reformulation of Peters' representability criterion as a nearest-integer parity test in the unit-cost model;

(N3) an exact admissible-parallelogram search procedure that can be implemented with integer arithmetic only, via row-wise binary search in the Minkowski embedding rather than floating-point root bounds;

(N4) a unit-square balancing step that reduces skewed inputs to the norm-sized regime before the finite search begins.

The canonical search algorithm stated in the paper is the same exact trace-enclosure plus sign/bisection routine used in the companion repository; there is no separate floating-point or idealised implementation layer hidden behind the analysis. Compared with the field algorithms in [3, 2, 9, 11, 10], the present problem is integral rather than field-valued: the input lies in the order $\mathcal{O}_D$ itself, the target is an integral decomposition, and the method no longer proceeds through a local–global principle or local square-class computations. Compared with [13, 12], the present paper is not about Waring numbers of henselian local rings or Pythagoras numbers of geometric rings, but about the exact minimal number of squares for a *given*

element of a fixed arithmetic order. In this sense the contribution is best viewed as a first explicit element-wise exact-length algorithm for maximal real quadratic orders. The key point is to separate two tasks. The computational section strengthens this algorithmic viewpoint with three reproducible experiments: a balancing ablation along Pell orbits, cross-field length distributions for several representative orders, and a bounded-trace brute-force validation sweep.

Software availability and reproducibility. The public repository associated with this paper will be maintained at https://github.com/tomaszkania/quadratic_sos. The submission bundle contains the same source tree. The package has no runtime dependency outside the Python standard library; optional extras are used only for tests, notebooks, and plot generation. The repository mirrors the exact-arithmetic architecture proved in the paper: search regions are enumerated by integer arithmetic only, the constructor rejects non-squarefree radicands D, and the exact-length interface is restricted to non-zero inputs. The scripts in `scripts/` regenerate the computational tables, validation outputs, and performance plots.

For a TOMS Algorithm-paper submission the software component is supplied as a separate archive in the repository's `submission/` directory. That archive contains the installable source package, non-interactive drivers, regression and validation tests, expected-output summaries, table and figure reproduction scripts, a manifest, and portability notes. The driver `scripts/run_all_checks.py` requires no interactive input and is intended as the first command for referees.

Representability. Since every sum of squares in $\mathcal{O}_D$ is a sum of at most five squares, Peters' five-square criterion decides whether α is representable at all. We rewrite that criterion as a nearest-integer test; this produces a decision procedure using a constant number of arithmetic operations in the unit-cost model.

Exact length. Once representability is known, every summand in any representation of α is bounded in both real embeddings by $\sqrt{\sigma_i(\alpha)}$. This gives a finite search region in the Minkowski embedding. After enumerating the corresponding finite set of relevant squares, the cases $\ell_D(\alpha) = 1, 2, 3, 4$ are decided by set-membership tests, and the remaining representable case is necessarily 5.

The arithmetic ingredients are classical; the new part is their algorithmic packaging. In particular, this paper provides:

(i) a representability test requiring only a constant number of arithmetic operations in the unit-cost model;

(ii) an exact finite-search algorithm for $\ell_D(\alpha) \in \{1, 2, 3, 4, 5, \infty\}$;

(iii) a corrected formulation of the search bounds, using $\Delta_D = |\omega_D - \omega'_D|$ rather than the incomplete expression $\sqrt{D}$;

(iv) a complexity discussion strengthened by exact unit-square balancing;

(v) an explicit algorithm computing $P(\mathcal{O}_D)$ and an integral Pythagoras element.

The broader context comes from universal quadratic forms and quadratic Waring invariants over totally real fields. The present problem is equivalent to deciding representability of a specific algebraic integer by the diagonal form $\langle 1, \dots, 1 \rangle$ of small rank, and therefore sits naturally inside the recent arithmetic of universal forms and indecomposables [6, 15].

The paper is organised as follows. Section 2 fixes notation and recalls the classical results of Peters and the quadratic Pythagoras classification. In Section 3 we derive the constant-cost

representability test. Section 4 constructs the finite search set and proves correctness of the exact length algorithm. Section 5 discusses explicit search-set bounds, unit-square balancing and reconstruction of minimal decompositions. Section 6 computes $P(\mathcal{O}_D)$ and produces a Pythagoras element. Section 7 contains an end-to-end worked example together with a computational study: a local table over $\mathcal{O}_{10}$, a balancing ablation on Pell orbits, cross-field distributions, and a bounded-trace validation sweep.

2. Preliminaries and classical input

Throughout the paper $D \geqslant 2$ is squarefree and $K_D = \mathbb{Q}(\sqrt{D})$ is a real quadratic field. Its ring of integers is

$$\mathcal{O}_D = \mathbb{Z}[\omega_D], \qquad \omega_D = \begin{cases} \sqrt{D}, & D \equiv 2,3 \pmod 4, \\ \dfrac{1+\sqrt{D}}{2}, & D \equiv 1 \pmod 4. \end{cases}$$

We write ω_D' for the Galois conjugate of ω_D, so

$$\omega_D' = \begin{cases} -\sqrt{D}, & D \equiv 2,3 \pmod 4, \\ \dfrac{1-\sqrt{D}}{2}, & D \equiv 1 \pmod 4. \end{cases}$$

The two real embeddings are denoted by

$$\sigma_1(a+b\omega_D) = a + b\omega_D, \qquad \sigma_2(a+b\omega_D) = a + b\omega_D'.$$

We also use the quantity

$$\Delta_D := |\omega_D - \omega_D'| = \begin{cases} 2\sqrt{D}, & D \equiv 2,3 \pmod 4, \\ \sqrt{D}, & D \equiv 1 \pmod 4, \end{cases}$$

which is the covolume of the Minkowski lattice $\sigma(\mathcal{O}_D) \subset \mathbb{R}^2$. Indeed, in the basis $(1,1)$ and (ω_D, ω_D') the absolute value of the determinant is exactly $|\omega_D - \omega_D'| = \Delta_D$.

Definition 2.1. For $0 \neq \alpha \in \mathcal{O}_D$, the *integral length* of α is

$$\ell_D(\alpha) = \min\{n \in \mathbb{N} : \alpha = x_1^2 + \cdots + x_n^2,\ x_i \in \mathcal{O}_D\},$$

if such a representation exists, and $\ell_D(\alpha) = \infty$ otherwise.

The *integral Pythagoras number* of $\mathcal{O}_D$ is

$$P(\mathcal{O}_D) = \sup\{\ell_D(\alpha) : \alpha \in \mathcal{O}_D,\ \ell_D(\alpha) < \infty\}.$$

An *integral Pythagoras element* is an element $a_D \in \mathcal{O}_D$ with $\ell_D(a_D) = P(\mathcal{O}_D)$.

We use the norm and trace

$$\mathrm{N}(a+b\omega_D) = \sigma_1(a+b\omega_D)\,\sigma_2(a+b\omega_D), \qquad \mathrm{Tr}(a+b\omega_D) = \sigma_1(a+b\omega_D) + \sigma_2(a+b\omega_D).$$

More explicitly,

$$\mathrm{N}(a+b\omega_D) = \begin{cases} a^2 - Db^2, & D \equiv 2,3 \pmod 4, \\ a^2 + ab - \dfrac{D-1}{4}b^2, & D \equiv 1 \pmod 4. \end{cases}$$

2.1. **Elementary observations.**

Lemma 2.2. *Let $0 \neq \alpha \in \mathcal{O}_D$.*

(a) *If α is a sum of squares in $\mathcal{O}_D$, then $\sigma_1(\alpha) > 0$ and $\sigma_2(\alpha) > 0$.*

(b) *If $D \equiv 2, 3 \pmod 4$ and $\alpha = a + b\sqrt{D}$ is a sum of squares in $\mathcal{O}_D = \mathbb{Z}[\sqrt{D}]$, then b is even.*

(c) *If α' denotes the Galois conjugate of α, then $\ell_D(\alpha') = \ell_D(\alpha)$.*

(d) *If $\varepsilon \in \mathcal{O}_D^\times$ is a unit, then $\ell_D(\varepsilon^2\alpha) = \ell_D(\alpha)$.*

Proof. If $\alpha = \sum_i x_i^2$, then for each embedding σ_j we have

$$\sigma_j(\alpha) = \sum_i \sigma_j(x_i)^2.$$

Since $\alpha \neq 0$, not all x_i vanish, so both sums are strictly positive. This proves (a).

If $D \equiv 2, 3 \pmod 4$ and $x = u + v\sqrt{D}$, then

$$x^2 = u^2 + Dv^2 + 2uv\sqrt{D},$$

so every square has an even coefficient at $\sqrt{D}$. This gives (b). Statements (c) and (d) follow by applying conjugation, respectively multiplication by ε, to a representation of α as a sum of squares. □

Remark 2.3. Lemma 2.2(d) provides a useful optimisation. Since $\mathcal{O}_D^\times = \{\pm\varepsilon_D^m : m \in \mathbb{Z}\}$ for a fundamental unit $\varepsilon_D > 1$, one may replace α by a suitable unit-square multiple $\varepsilon_D^{2k}\alpha$ before running the finite search. This does not change the answer and can make the two real embeddings comparable; we exploit this in Section 5.

2.2. **Two classical theorems.** The first input is Peters' characterisation of sums of squares in real quadratic orders. We use the statement in the form quoted by Raška in modern notation [19, Theorem 7]; for the original sources see Peters' two papers [17, 18].

Theorem 2.4 (Peters)**.** *Let $0 \neq \alpha \in \mathcal{O}_D$ be totally positive.*

(i) *Assume $D \equiv 1 \pmod 4$ and write $\alpha = a + b\omega_D$ with $a, b \in \mathbb{Z}$. Then α is a sum of squares in $\mathcal{O}_D$ if and only if there exists $c \in \mathbb{Z}$ with $c \equiv b \pmod 2$ and*

$$\frac{2a + b - 2\sqrt{\mathrm{N}(\alpha)}}{D} \leqslant c \leqslant \frac{2a + b + 2\sqrt{\mathrm{N}(\alpha)}}{D}.$$

(ii) *Assume $D \equiv 2, 3 \pmod 4$ and write $\alpha = a + b\sqrt{D}$ with $a, b \in \mathbb{Z}$. Then α is a sum of squares in $\mathcal{O}_D$ if and only if b is even and there exists $c \in \mathbb{Z}$ such that*

$$\frac{a - \sqrt{\mathrm{N}(\alpha)}}{2D} \leqslant c \leqslant \frac{a + \sqrt{\mathrm{N}(\alpha)}}{2D}.$$

The second input is the exact classification of Pythagoras numbers of maximal real quadratic orders. In the form used here it is collected in [14, Theorem 3.1]; the underlying classical sources are Cohn–Pall, Maaß, Peters and Scharlau [1, 16, 18, 20].

Theorem 2.5. *For the maximal order $\mathcal{O}_D$ of $K_D = \mathbb{Q}(\sqrt{D})$ one has*

$$P(\mathcal{O}_D) = \begin{cases} 3, & D \in \{2, 3, 5\}, \\ 4, & D \in \{6, 7\}, \\ 5, & \text{otherwise.} \end{cases}$$

In particular, every representable element of $\mathcal{O}_D$ is a sum of at most five squares.

Corollary 2.6. *For every nonzero $\alpha \in \mathcal{O}_D$,*

$$\ell_D(\alpha) \in \{1, 2, 3, 4, 5, \infty\},$$

and α is a sum of squares if and only if it is a sum of five squares.

Proof. This is immediate from Theorem 2.5. □

3. Representability by integral squares

Peters' criterion already decides representability. For computation it is convenient to turn the interval condition into a nearest-integer test and to state carefully what is meant by "constant-cost".

Proposition 3.1. *Let $0 \neq \alpha \in \mathcal{O}_D$.*

(i) *Assume $D \equiv 1 \pmod 4$ and write $\alpha = a + b\omega_D$. Set $C = 2a + b$ and let*

$$\mathcal{A}_b = \{c \in \mathbb{Z} : c \equiv b \pmod 2\}.$$

Choose $c_0 \in \mathcal{A}_b$ minimising $|Dc - C|$. Then α is a sum of squares in $\mathcal{O}_D$ if and only if

$$\sigma_1(\alpha) > 0, \qquad \sigma_2(\alpha) > 0, \qquad \text{and} \qquad (Dc_0 - C)^2 \leqslant 4\,\mathrm{N}(\alpha).$$

(ii) *Assume $D \equiv 2, 3 \pmod 4$ and write $\alpha = a + b\sqrt{D}$. Choose $c_0 \in \mathbb{Z}$ minimising $|2Dc - a|$. Then α is a sum of squares in $\mathcal{O}_D$ if and only if*

$$\sigma_1(\alpha) > 0, \qquad \sigma_2(\alpha) > 0, \qquad b \equiv 0 \pmod 2, \qquad \text{and} \qquad (2Dc_0 - a)^2 \leqslant \mathrm{N}(\alpha).$$

Moreover, the test uses a constant number of arithmetic operations and comparisons in the unit-cost model.

Proof. By Corollary 2.6, α is representable if and only if it is a sum of five squares. If α is representable, then Lemma 2.2(a) gives total positivity, and in the second case Lemma 2.2(b) gives the parity condition.

Assume first that $D \equiv 1 \pmod 4$. By Theorem 2.4, representability is equivalent to the existence of an integer $c \equiv b \pmod 2$ satisfying

$$|Dc - C| \leqslant 2\sqrt{\mathrm{N}(\alpha)}.$$

Among all such integers, the left-hand side is minimised at a nearest admissible integer c_0. Hence such a c exists if and only if the minimum itself satisfies the same inequality, which is exactly

$$(Dc_0 - C)^2 \leqslant 4\,\mathrm{N}(\alpha).$$

The proof in the case $D \equiv 2, 3 \pmod 4$ is identical. By Theorem 2.4, representability is equivalent to b even and the existence of an integer c with

$$|2Dc - a| \leqslant \sqrt{\mathrm{N}(\alpha)}.$$

The minimum is attained at a nearest integer c_0 to $a/(2D)$, and squaring gives the desired criterion. The final assertion is immediate: the procedure uses only a constant number of arithmetic operations on the coefficients of α and D, together with exact comparisons. □

Remark 3.2. Proposition 3.1 is "constant-cost" only in the unit-cost arithmetic model. In the bit model the input size still matters, of course: one performs a constant number of integer additions, multiplications, congruence tests and comparisons on integers whose bit size is $O(\log|D| + \log H)$, where H bounds the coefficients of α.

Algorithm 1: Representability in $\mathcal{O}_D$

Input : A squarefree integer $D \geqslant 2$ and a nonzero element $\alpha \in \mathcal{O}_D$.
Output : `TRUE` if α is a sum of squares in $\mathcal{O}_D$, otherwise `FALSE`.

if $\sigma_1(\alpha) \leqslant 0$ *or* $\sigma_2(\alpha) \leqslant 0$ **then**
  return `FALSE`;
if $D \equiv 1 \pmod 4$ **then**
  Write $\alpha = a + b\omega_D$ with $a, b \in \mathbb{Z}$ and set $C \leftarrow 2a + b$;
  Let c_0 be a nearest integer to C/D with $c_0 \equiv b \pmod 2$;
  if $(Dc_0 - C)^2 \leqslant 4\,\mathrm{N}(\alpha)$ **then**
    return `TRUE`;
  return `FALSE`;
else
  Write $\alpha = a + b\sqrt{D}$ with $a, b \in \mathbb{Z}$;
  if b *is odd* **then**
    return `FALSE`;
  Let c_0 be a nearest integer to $a/(2D)$;
  if $(2Dc_0 - a)^2 \leqslant \mathrm{N}(\alpha)$ **then**
    return `TRUE`;
  return `FALSE`;

Theorem 3.3. *Algorithm 1 is correct.*

Proof. This is Proposition 3.1 written in pseudocode. □

4. Finite search and exact integral length

We now assume that $\alpha \in \mathcal{O}_D$ is totally positive and construct an explicit finite search set containing every possible summand of every representation of α.

4.1. Corrected embedding bounds.

Set

$$A_1 = \sqrt{\sigma_1(\alpha)}, \qquad A_2 = \sqrt{\sigma_2(\alpha)}.$$

For $x \in K_D$ we write $x^2 \preccurlyeq \alpha$ if $\sigma_i(x^2) \leqslant \sigma_i(\alpha)$ for $i = 1, 2$.

Lemma 4.1. *Let $x = u + v\omega_D \in \mathcal{O}_D$ and assume $x^2 \preccurlyeq \alpha$. Then*

$$|v| \leqslant \frac{A_1 + A_2}{\Delta_D}$$

and

$$|u| \leqslant \frac{|\omega_D|A_2 + |\omega_D'|A_1}{\Delta_D}.$$

In particular, only finitely many elements $x \in \mathcal{O}_D$ satisfy $x^2 \preccurlyeq \alpha$.

Proof. From $x^2 \preccurlyeq \alpha$ we obtain

$$|\sigma_1(x)| \leqslant A_1, \qquad |\sigma_2(x)| \leqslant A_2.$$

Since

$$\sigma_1(x) - \sigma_2(x) = v(\omega_D - \omega'_D),$$

we get

$$|v| = \left| \frac{\sigma_1(x) - \sigma_2(x)}{\omega_D - \omega'_D} \right| \leqslant \frac{|\sigma_1(x)| + |\sigma_2(x)|}{|\omega_D - \omega'_D|} \leqslant \frac{A_1 + A_2}{\Delta_D}.$$

This is the point at which the distinction between $D \equiv 1 \pmod 4$ and $D \equiv 2, 3 \pmod 4$ matters: one has $\Delta_D = \sqrt{D}$ in the former case and $\Delta_D = 2\sqrt{D}$ in the latter.

Next, solving the linear system

$$\sigma_1(x) = u + v\omega_D, \qquad \sigma_2(x) = u + v\omega'_D$$

for u yields

$$u = \frac{\omega_D \sigma_2(x) - \omega'_D \sigma_1(x)}{\omega_D - \omega'_D}.$$

Therefore

$$|u| \leqslant \frac{|\omega_D|\,|\sigma_2(x)| + |\omega'_D|\,|\sigma_1(x)|}{|\omega_D - \omega'_D|} \leqslant \frac{|\omega_D| A_2 + |\omega'_D| A_1}{\Delta_D}.$$

Since $u, v \in \mathbb{Z}$, only finitely many pairs (u, v) are possible. □

Corollary 4.2. *Define*

$$U_\alpha := \left\lfloor \frac{|\omega_D| A_2 + |\omega'_D| A_1}{\Delta_D} \right\rfloor, \qquad V_\alpha := \left\lfloor \frac{A_1 + A_2}{\Delta_D} \right\rfloor.$$

Then every $x = u + v\omega_D \in \mathcal{O}_D$ with $x^2 \preccurlyeq \alpha$ satisfies $|u| \leqslant U_\alpha$ and $|v| \leqslant V_\alpha$.

Definition 4.3. For totally positive $\alpha \in \mathcal{O}_D$ define the finite search set

$$B(\alpha) = \{x \in \mathcal{O}_D : x^2 \preccurlyeq \alpha\}.$$

Let

$$S(\alpha) = \{x^2 : x \in B(\alpha)\} \subseteq \mathcal{O}_D$$

be the corresponding set of relevant squares, and define

$$T_2(\alpha) = \{s_1 + s_2 : s_1, s_2 \in S(\alpha)\}.$$

By Corollary 4.2, the set $B(\alpha)$ is finite.

Proposition 4.4 (Geometric row description)**.** *For each integer v, define*

$$u_{\min}(v) = \lceil \max\{-A_1 - v\omega_D,\, -A_2 - v\omega'_D\} \rceil$$

and

$$u_{\max}(v) = \lfloor \min\{A_1 - v\omega_D,\ A_2 - v\omega'_D\} \rfloor .$$

Then

$$\{u \in \mathbb{Z} : u + v\omega_D \in B(\alpha)\} = \{u \in \mathbb{Z} : u_{\min}(v) \leqslant u \leqslant u_{\max}(v)\}.$$

In particular, each fixed-v row of $B(\alpha)$ is an interval of integers or is empty.

Proof. The inequalities $|\sigma_1(u+v\omega_D)| \leqslant A_1$ and $|\sigma_2(u+v\omega_D)| \leqslant A_2$ are equivalent to

$$-A_1 - v\omega_D \leqslant u \leqslant A_1 - v\omega_D$$

and

$$-A_2 - v\omega'_D \leqslant u \leqslant A_2 - v\omega'_D.$$

Hence an integer u is admissible if and only if it lies in the intersection of these two intervals, which is exactly the range $u_{\min}(v) \leqslant u \leqslant u_{\max}(v)$. □

Proposition 4.5 (Exact trace enclosure for a fixed row)**.** *Let $\alpha \in \mathcal{O}_D$ be totally positive and fix $v \in \mathbb{Z}$.*

(i) *If $D \equiv 2,3 \pmod 4$ and $x = u + v\sqrt{D} \in B(\alpha)$, then*

$$u^2 \leqslant \frac{\mathrm{Tr}(\alpha)}{2} - Dv^2.$$

Hence the admissible integers u lie in the interval

$$I_v^{\mathrm{tr}}(\alpha) = \left[-\left\lfloor \sqrt{\frac{\mathrm{Tr}(\alpha)}{2} - Dv^2} \right\rfloor , \left\lfloor \sqrt{\frac{\mathrm{Tr}(\alpha)}{2} - Dv^2} \right\rfloor \right]$$

whenever $\mathrm{Tr}(\alpha)/2 - Dv^2 \geqslant 0$, and the row is empty otherwise.

(ii) *If $D \equiv 1 \pmod 4$ and $x = u + v\omega_D \in B(\alpha)$, then*

$$(2u+v)^2 \leqslant 2\,\mathrm{Tr}(\alpha) - Dv^2.$$

Hence the admissible integers u lie in the interval

$$I_v^{\mathrm{tr}}(\alpha) = \left[\left\lceil \frac{-v - \left\lfloor \sqrt{2\,\mathrm{Tr}(\alpha) - Dv^2} \right\rfloor}{2} \right\rceil , \left\lfloor \frac{-v + \left\lfloor \sqrt{2\,\mathrm{Tr}(\alpha) - Dv^2} \right\rfloor}{2} \right\rfloor \right]$$

whenever $2\,\mathrm{Tr}(\alpha) - Dv^2 \geqslant 0$, and the row is empty otherwise.

In particular, rows can occur only for

$$|v| \leqslant V_\alpha^{\mathrm{tr}} := \begin{cases} \left\lfloor \sqrt{\dfrac{2\,\mathrm{Tr}(\alpha)}{D}} \right\rfloor, & D \equiv 1 \pmod 4, \\ \left\lfloor \sqrt{\dfrac{\mathrm{Tr}(\alpha)}{2D}} \right\rfloor, & D \equiv 2,3 \pmod 4. \end{cases}$$

Proof. Write $x = u + v\omega_D$ or $x = u + v\sqrt{D}$ according to the congruence class of D modulo 4. Since $x^2 \preccurlyeq \alpha$, one has $\mathrm{Tr}(x^2) \leqslant \mathrm{Tr}(\alpha)$. If $D \equiv 2,3 \pmod 4$, then

$$\mathrm{Tr}\big((u+v\sqrt{D})^2\big) = 2u^2 + 2Dv^2,$$

so $2u^2+2Dv^2 \leqslant \mathrm{Tr}(\alpha)$, which is exactly (i). If $D \equiv 1 \pmod 4$, then using $\omega_D^2 = (D-1)/4 + \omega_D$ and $\mathrm{Tr}(a+b\omega_D) = 2a+b$,

$$\mathrm{Tr}\big((u+v\omega_D)^2\big) = 2u^2 + 2uv + \frac{D+1}{2}v^2.$$

Multiplying by 2 and completing the square gives

$$(2u+v)^2 + Dv^2 \leqslant 2\,\mathrm{Tr}(\alpha),$$

which is equivalent to (ii). The bound on $|v|$ is immediate in both cases. □

Proposition 4.6 (Exact row search by directed bisection)**.** *Let $\alpha \in \mathcal{O}_D$ be totally positive and fix $v \in \mathbb{Z}$. Set*

$$J_v(\alpha) = \{u \in I_v^{\mathrm{tr}}(\alpha) : u + v\omega_D \in B(\alpha)\}.$$

Then $J_v(\alpha)$ is an interval of integers or is empty. Moreover:

(a) *one can determine whether $J_v(\alpha)$ is empty and, if not, find one admissible integer $u_0 \in J_v(\alpha)$ in $O(\log(1 + |I_v^{\mathrm{tr}}(\alpha)|))$ exact arithmetic operations;*

(b) *once such a u_0 is known, the left and right endpoints of $J_v(\alpha)$ are found by two further binary searches of the same complexity;*

(c) *consequently, $B(\alpha)$ and $S(\alpha)$ can be enumerated exactly, row by row, without any floating-point arithmetic.*

Proof. By Proposition 4.4, the admissible integers on a fixed row already form an interval or are empty, so only the exact search remains to justify. For a trial value $u \in I_v^{\mathrm{tr}}(\alpha)$ put $x = u + v\omega_D$. The admissibility test $x \in B(\alpha)$ is equivalent to

$$\sigma_1(\alpha - x^2) \geqslant 0 \quad \text{and} \quad \sigma_2(\alpha - x^2) \geqslant 0,$$

which is decidable by exact sign tests on elements of $\mathcal{O}_D$. Assume that $x \notin B(\alpha)$. For each violated embedding $i \in \{1, 2\}$ one has $|\sigma_i(x)| > A_i$. Because the map $u \mapsto \sigma_i(u + v\omega_D)$ is affine with slope 1, the sign of $\sigma_i(x)$ determines on which side of the admissible interval for the i-th embedding the point lies: if $\sigma_i(x) < 0$, then every admissible integer lies strictly to the right of u; if $\sigma_i(x) > 0$, then every admissible integer lies strictly to the left. These signs are again computable exactly. If the two violated embeddings prescribe opposite directions, then the two embedding intervals are disjoint and $J_v(\alpha) = \varnothing$; otherwise the common prescribed half-interval contains all admissible points. Thus a directed binary search halves the current search interval at each step until either an admissible point is found or emptiness is certified. This proves (a).

Once one admissible point u_0 is found, the interval property of $J_v(\alpha)$ allows one to locate its left and right endpoints by ordinary binary search using only the admissibility test. This proves (b), and (c) follows by scanning all integers between the two exact endpoints on every nonempty row. □

Remark 4.7. The geometric description in Proposition 4.4 is the two-dimensional analogue of enumerating lattice points inside the actual search body rather than its axis-aligned bounding box. In higher-dimensional lattice algorithms this viewpoint is standard in Fincke–Pohst style enumeration [5]; the point of Proposition 4.6 is that in the present dimension the same idea admits a completely exact implementation by trace enclosures and integer binary search.

Lemma 4.8 (Completeness of the search set)**.** *Let $\alpha \in \mathcal{O}_D$ be totally positive and suppose*

$$\alpha = x_1^2 + \cdots + x_n^2 \qquad (x_i \in \mathcal{O}_D).$$

Then each x_i belongs to $B(\alpha)$, and hence each square x_i^2 belongs to $S(\alpha)$.

Proof. For each embedding σ_j and each index i one has

$$\sigma_j(\alpha) = \sum_{k=1}^{n} \sigma_j(x_k)^2 \geqslant \sigma_j(x_i)^2.$$

Therefore $|\sigma_j(x_i)| \leqslant A_j$ for $j = 1, 2$, so Lemma 4.1 applies and shows that $x_i \in B(\alpha)$. □

4.2. **Exact-length criterion.**

Theorem 4.9. *Let $0 \neq \alpha \in \mathcal{O}_D$.*

(i) $\ell_D(\alpha) = 1$ *if and only if* $\alpha \in S(\alpha)$.
(ii) $\ell_D(\alpha) = 2$ *if and only if* $\alpha \notin S(\alpha)$ *and* $\alpha \in T_2(\alpha)$.
(iii) $\ell_D(\alpha) = 3$ *if and only if* $\alpha \notin S(\alpha) \cup T_2(\alpha)$ *and there exists* $s \in S(\alpha)$ *with* $\alpha - s \in T_2(\alpha)$.
(iv) $\ell_D(\alpha) = 4$ *if and only if none of the previous cases occurs and there exists* $t \in T_2(\alpha)$ *with* $\alpha - t \in T_2(\alpha)$.
(v) *If Algorithm 1 returns* `TRUE` *and none of* (i)–(iv) *holds, then* $\ell_D(\alpha) = 5$.
(vi) *If Algorithm 1 returns* `FALSE`, *then* $\ell_D(\alpha) = \infty$.

Proof. If Algorithm 1 returns `FALSE`, then by Theorem 3.3 the element α is not a sum of squares in $\mathcal{O}_D$, so $\ell_D(\alpha) = \infty$. This proves (vi).

Assume from now on that Algorithm 1 returns `TRUE`. By Corollary 2.6, the remaining possibilities are $1, 2, 3, 4, 5$.

Assertion (i) is immediate from the definition of $S(\alpha)$.

For (ii), if $\alpha = s_1 + s_2$ is a sum of two squares, then Lemma 4.8 gives $s_1, s_2 \in S(\alpha)$, hence $\alpha \in T_2(\alpha)$. Conversely, any identity $\alpha = s_1 + s_2$ with $s_1, s_2 \in S(\alpha)$ is by definition a representation by two squares. Excluding the case $\alpha \in S(\alpha)$ forces the minimal length to be exactly 2.

For (iii), a representation by three squares has the form $\alpha = s_1 + s_2 + s_3$ with $s_i \in S(\alpha)$ by Lemma 4.8. Equivalently, $\alpha - s_1 = s_2 + s_3 \in T_2(\alpha)$ for some $s_1 \in S(\alpha)$. Conversely, any such identity yields a representation by three squares. Excluding lengths 1 and 2 forces the minimal length to be exactly 3.

For (iv), if $\alpha = s_1 + s_2 + s_3 + s_4$ with each $s_i \in S(\alpha)$, then $\alpha = (s_1 + s_2) + (s_3 + s_4) = t_1 + t_2$ with $t_1, t_2 \in T_2(\alpha)$. Conversely, if $\alpha = t_1 + t_2$ with $t_1, t_2 \in T_2(\alpha)$, then by definition there exist $s_1, s_2, s_3, s_4 \in S(\alpha)$ such that $t_1 = s_1 + s_2$ and $t_2 = s_3 + s_4$, so $\alpha = s_1 + s_2 + s_3 + s_4$ is a representation by four squares. Excluding the previous cases forces the minimal length to be exactly 4.

Finally, if none of (i)–(iv) holds but α is representable, then the only remaining possibility allowed by Corollary 2.6 is $\ell_D(\alpha) = 5$. This proves (v). □

Theorem 4.10. *Algorithm 2 (below) is correct. Its core search uses only exact integer arithmetic and exact sign tests in $\mathcal{O}_D$; in particular, no floating-point approximation is required.*

Proof. Proposition 4.5 supplies a finite family of rows containing all possible elements of $B(\alpha)$. On each row, Proposition 4.6 recovers the exact admissible interval of integers u, so the algorithm inserts precisely the squares x^2 with $x \in B(\alpha)$. The final assertion is part (c) of Proposition 4.6. □

Algorithm 2: Enumerating the relevant squares by exact row search

Input : A squarefree integer $D \geqslant 2$ and a totally positive element $\alpha \in \mathcal{O}_D$.
Output : The finite set $S(\alpha)$ of all squares x^2 with $x^2 \preccurlyeq \alpha$.

Compute the trace row bound V_α^{tr} from Proposition 4.5;
Initialise $S \leftarrow \varnothing$;
for $v = -V_\alpha^{\mathrm{tr}}$ **to** V_α^{tr} **do**
    Compute the trace enclosure $I_v^{\mathrm{tr}}(\alpha)$ from Proposition 4.5;
    if $I_v^{\mathrm{tr}}(\alpha)$ *is empty* **then**
        continue;
    Use Proposition 4.6 to determine the exact admissible row

$$J_v(\alpha) = \{u \in I_v^{\mathrm{tr}}(\alpha) : (u + v\omega_D)^2 \preccurlyeq \alpha\}.$$

    if $J_v(\alpha)$ *is empty* **then**
        continue;
    Let $u_{\min}(v) \leqslant u_{\max}(v)$ be the two endpoints of $J_v(\alpha)$;
    for $u = u_{\min}(v)$ **to** $u_{\max}(v)$ **do**
        Insert $(u + v\omega_D)^2$ into S;
return S;

Theorem 4.11. *Algorithm 3 (below) is correct.*

Proof. Algorithm 1 decides representability by Theorem 3.3. If it answers negatively, the output ∞ is correct.

Assume from now on that α is representable, and let $\beta = u^{2k}\alpha$ be the balanced unit-square multiple computed in the second line. By Lemma 2.2(d), $\ell_D(\beta) = \ell_D(\alpha)$. By Theorem 4.10, Algorithm 2 returns the exact set $S(\beta)$. The test $\beta \in S(\beta)$ is correct by Theorem 4.9(i). Next, β has length 2 if and only if $\beta = s_1 + s_2$ for some $s_1, s_2 \in S(\beta)$; equivalently, there exists $s \in S(\beta)$ with $\beta - s \in S(\beta)$. Thus the second loop decides the case $\ell_D(\beta) = 2$ and hence $\ell_D(\alpha) = 2$.

If $p = P(\mathcal{O}_D) = 3$, then every representable element has length at most 3 by Theorem 2.5, so after excluding lengths 1 and 2 the algorithm may correctly return 3. Assume therefore that $p \geqslant 4$. The next loop checks whether $\beta - s \in T_2(\beta)$ for some $s \in S(\beta)$, which is equivalent to $\ell_D(\beta) = 3$ by Theorem 4.9(iii) and therefore to $\ell_D(\alpha) = 3$.

If $p = 4$, then every representable element has length at most 4, so after excluding lengths $1, 2, 3$ the algorithm may correctly return 4. Finally, when $p = 5$, the last loop is exactly the four-square test from Theorem 4.9(iv) applied to β, and failure of that test forces $\ell_D(\beta) = 5$ by Theorem 4.9(v). Since $\ell_D(\beta) = \ell_D(\alpha)$, the returned value is correct. □

Algorithm 3: Exact integral length in $\mathcal{O}_D$

Input : A squarefree integer $D \geqslant 2$ and a nonzero element $\alpha \in \mathcal{O}_D$.
Output : The exact value of $\ell_D(\alpha)$ in $\{1, 2, 3, 4, 5, \infty\}$.

if *Algorithm 1 returns* `FALSE` **then**
  return ∞;
Compute a balanced unit-square multiple $\beta = u^{2k}\alpha$ as in Lemma 5.4;
Determine $p \leftarrow P(\mathcal{O}_D)$ from Theorem 2.5;
Compute $S \leftarrow S(\beta)$ using Algorithm 2 and store it as a hash set;
if $\beta \in S$ **then**
  return 1;
foreach $s \in S$ **do**
  if $\beta - s \in S$ **then**
    return 2;
if $p = 3$ **then**
  return 3;
Construct the pair-sum set

$$T_2 \leftarrow \{s_1 + s_2 : s_1, s_2 \in S\}$$

and store it as a hash set;
foreach $s \in S$ **do**
  if $\beta - s \in T_2$ **then**
    return 3;
if $p = 4$ **then**
  return 4;
foreach $t \in T_2$ **do**
  if $\beta - t \in T_2$ **then**
    return 4;
return 5;

Remark 4.12 (Ternary short-circuits and Maaß's criterion)**.** The short-circuit at $p = 3$ removes the construction of $T_2(\beta)$ in the cases $D \in \{2, 3, 5\}$. In particular, for $D = 5$ one could also replace the uniform three-square step by Maaß's specialised ternary criterion [16]. We keep the present uniform algorithm because it applies unchanged to every maximal real quadratic order.

5. Complexity, balancing, and constructive output

We first state the cost of the search-and-decision phase after balancing, then derive explicit bounds for the size of the resulting finite square set, and finally explain how to recover an actual minimal decomposition.

Theorem 5.1. *Let $\alpha \in \mathcal{O}_D$ be representable and let $\beta = u^{2k}\alpha$ be the balanced unit-square multiple supplied to the search-and-decision phase of Algorithm 3. Put*

$$M(\beta) = |S(\beta)|, \qquad N_B(\beta) = |B(\beta)|, \qquad \text{and} \qquad \Sigma_{\mathrm{row}}(\beta) = \sum_{|v| \leqslant V_\beta^{\mathrm{tr}}} \log(1 + |I_v^{\mathrm{tr}}(\beta)|),$$

where V_β^{tr} and the trace row intervals $I_v^{\mathrm{tr}}(\beta)$ are given by Proposition 4.5. Then the search-and-decision phase of Algorithm 3 after balancing can be implemented as follows.

(a) *By Algorithm 2, one enumerates $B(\beta)$ and builds the hash set $S(\beta)$ in*

$$O\big(N_B(\beta) + \Sigma_{\mathrm{row}}(\beta)\big)$$

exact arithmetic operations and $O(M(\beta))$ memory.

(b) *The tests for lengths 1 and 2 require one hash lookup, respectively $M(\beta)$ hash lookups in $S(\beta)$, so they take expected time $O(1)$ and $O(M(\beta))$ after $S(\beta)$ has been built.*

(c) *If $P(\mathcal{O}_D) \geqslant 4$, one constructs $T_2(\beta)$ in expected time $O(M(\beta)^2)$ and memory $O(M(\beta)^2)$. The subsequent three-square test uses only $M(\beta)$ additional hash lookups in $T_2(\beta)$.*

(d) *If $P(\mathcal{O}_D) = 5$, the four-square test scans $T_2(\beta)$ and performs $|T_2(\beta)| \leqslant M(\beta)^2$ further hash lookups in $T_2(\beta)$.*

Consequently, after the balanced representative β has been computed, the remaining search-and-decision phase uses

$$\text{time} = O\big(N_B(\beta) + \Sigma_{\mathrm{row}}(\beta) + M(\beta)^2\big), \qquad \text{memory} = O\big(M(\beta)^2\big)$$

in expected time with hash tables. In the ternary cases $D \in \{2, 3, 5\}$, the pair-sum set $T_2(\beta)$ need not be constructed, so the expected running time improves to

$$O\big(N_B(\beta) + \Sigma_{\mathrm{row}}(\beta) + M(\beta)\big).$$

The representability test of Algorithm 1 contributes only a constant number of arithmetic operations in the unit-cost model. In the present implementation, the balancing step itself is carried out iteratively and contributes an additional $O(|k|)$ exact unit multiplications and ratio comparisons, where $\beta = u^{2k}\alpha$.

Proof. Algorithm 2 scans exactly the $N_B(\beta)$ admissible roots and, by Proposition 4.6, performs on each row one directed bisection together with two boundary bisections. This gives the cost $O\big(N_B(\beta) + \Sigma_{\mathrm{row}}(\beta)\big)$ for building $S(\beta)$. The tests for lengths 1 and 2 use only membership in the hash set $S(\beta)$, so their costs are immediate. Once $T_2(\beta)$ is built, the three-square test scans $S(\beta)$ and checks whether $\beta - s \in T_2(\beta)$ for each $s \in S(\beta)$, while the four-square test scans $T_2(\beta)$ and checks whether $\beta - t \in T_2(\beta)$ for each $t \in T_2(\beta)$. This yields the stated time and memory bounds. In the ternary cases the short-circuit in Algorithm 3 avoids the construction of $T_2(\beta)$ altogether. □

Remark 5.2 (Space reduction by sorted pair-sum streams)**.** The explicit construction of $T_2(\beta)$ is convenient but uses quadratic memory. Since the four-square step is a collision problem between two pair-sum streams, one may instead adapt the classical Schroeppel–Shamir meet-in-the-middle technique [21] to lower the memory requirement to $O(M(\beta))$, at the cost of sorted-stream bookkeeping and an extra logarithmic factor in time. We retain the simpler explicit construction of $T_2(\beta)$ in the main algorithm.

Proposition 5.3. *For every totally positive $\alpha \in \mathcal{O}_D$ one has*

$$M(\alpha) \leqslant |B(\alpha)| \leqslant (2U_\alpha + 1)(2V_\alpha + 1),$$

where U_α and V_α are given by Corollary 4.2. In particular,

$$M(\alpha) = O(U_\alpha V_\alpha).$$

Proof. The inequality $M(\alpha) \leqslant |B(\alpha)|$ is immediate from the definition of $S(\alpha)$. The second inequality follows because there are at most $2U_\alpha + 1$ admissible integers u and at most $2V_\alpha + 1$ admissible integers v. □

The bound in Proposition 5.3 is fully explicit, but it depends on the imbalance of the two embeddings. For a fixed real quadratic ring one can sharpen the complexity statement by using the unit-square invariance from Lemma 2.2(d).

Lemma 5.4. *Fix a fundamental unit $\varepsilon_D > 1$ of $\mathcal{O}_D$. For every totally positive $\alpha \in \mathcal{O}_D$ there exists $k \in \mathbb{Z}$ such that*

$$\beta = \varepsilon_D^{2k}\alpha$$

satisfies

$$\varepsilon_D^{-2} \leqslant \frac{\sigma_1(\beta)}{\sigma_2(\beta)} < \varepsilon_D^2.$$

Moreover, $\ell_D(\beta) = \ell_D(\alpha)$ and $\mathrm{N}(\beta) = \mathrm{N}(\alpha)$.

Proof. Set

$$r = \frac{\sigma_1(\alpha)}{\sigma_2(\alpha)} > 0.$$

Since $\sigma_1(\varepsilon_D^2) = \varepsilon_D^2$ and $\sigma_2(\varepsilon_D^2) = \varepsilon_D^{-2}$, multiplying by ε_D^{2k} changes the ratio by a factor of ε_D^{4k}. Choose $k \in \mathbb{Z}$ so that

$$\varepsilon_D^{-2} \leqslant r\varepsilon_D^{4k} < \varepsilon_D^2.$$

This is possible because the intervals $[\varepsilon_D^{4m-2}, \varepsilon_D^{4m+2})$, $m \in \mathbb{Z}$, cover $(0, \infty)$. The equality of lengths is Lemma 2.2(d), and $\mathrm{N}(\beta) = \mathrm{N}(\varepsilon_D)^{2k}\,\mathrm{N}(\alpha) = \mathrm{N}(\alpha)$ because $\mathrm{N}(\varepsilon_D) = \pm 1$. □

Remark 5.5. For the complexity statements it is natural to use a fundamental unit, since this gives the smallest balancing constant. For the implementation, however, any fixed totally positive unit $u > 1$ suffices: replacing ε_D by u leaves the length invariant and yields the same balancing mechanism, with constants depending on u. The companion repository uses a positive norm-one Pell unit obtained from the continued fraction of $\sqrt{D}$, which keeps the balancing step exact while avoiding floating-point logarithms.

Corollary 5.6. *Let $\beta = \varepsilon_D^{2k}\alpha$ be balanced as in Lemma 5.4. Then*

$$A_1(\beta), A_2(\beta) \leqslant \varepsilon_D^{1/2}\,\mathrm{N}(\alpha)^{1/4},$$

and consequently

$$U_\beta \leqslant \varepsilon_D^{1/2}\,\mathrm{N}(\alpha)^{1/4}, \qquad V_\beta \leqslant \frac{2\varepsilon_D^{1/2}}{\Delta_D}\,\mathrm{N}(\alpha)^{1/4}.$$

In particular,

$$M(\beta) = O_D\left(\frac{\sqrt{\mathrm{N}(\alpha)}}{\Delta_D}\right),$$

and, once the balanced representative β has been computed, the search-and-decision phase of Algorithm 3 runs in expected time

$$O_D\left(\frac{\mathrm{N}(\alpha)}{\Delta_D^2}\right) = O_D\left(\frac{\mathrm{N}(\alpha)}{D}\right)$$

in the unit-cost model. The current implementation reaches β by iterative Pell-unit balancing and therefore adds $O(|k|)$ exact preprocessing steps, where $\beta = \varepsilon_D^{2k}\alpha$.

Proof. Put $r = \sigma_1(\beta)/\sigma_2(\beta)$. By Lemma 5.4, both r and r^{-1} are at most ε_D^2. Since β is totally positive, $\mathrm{N}(\beta) = \mathrm{N}(\alpha) > 0$, and therefore

$$\sigma_1(\beta)^2 = \mathrm{N}(\alpha)\, r, \qquad \sigma_2(\beta)^2 = \mathrm{N}(\alpha)\, r^{-1}.$$

Taking positive square roots gives

$$\sigma_1(\beta) = \mathrm{N}(\alpha)^{1/2} r^{1/2}, \qquad \sigma_2(\beta) = \mathrm{N}(\alpha)^{1/2} r^{-1/2},$$

whence

$$A_1(\beta) = \sigma_1(\beta)^{1/2} \leqslant \varepsilon_D^{1/2}\, \mathrm{N}(\alpha)^{1/4}, \qquad A_2(\beta) = \sigma_2(\beta)^{1/2} \leqslant \varepsilon_D^{1/2}\, \mathrm{N}(\alpha)^{1/4}.$$

Now $|\omega_D| + |\omega_D'| = \Delta_D$ in both congruence classes modulo 4, so Corollary 4.2 gives

$$U_\beta \leqslant \frac{|\omega_D| A_2(\beta) + |\omega_D'| A_1(\beta)}{\Delta_D} \leqslant \varepsilon_D^{1/2}\, \mathrm{N}(\alpha)^{1/4}$$

and

$$V_\beta \leqslant \frac{A_1(\beta) + A_2(\beta)}{\Delta_D} \leqslant \frac{2\varepsilon_D^{1/2}}{\Delta_D}\, \mathrm{N}(\alpha)^{1/4}.$$

The estimate for $M(\beta)$ follows from Proposition 5.3. Moreover,

$$\sigma_1(\beta), \sigma_2(\beta) \leqslant \varepsilon_D\, \mathrm{N}(\alpha)^{1/2}, \qquad \mathrm{Tr}(\beta) = O_D(\mathrm{N}(\alpha)^{1/2}).$$

Therefore Proposition 4.5 gives

$$V_\beta^{\mathrm{tr}} = O_D\left(\frac{\mathrm{N}(\alpha)^{1/4}}{\sqrt{D}}\right),$$

while every trace row interval has length

$$O(U_\beta) = O_D(\mathrm{N}(\alpha)^{1/4}).$$

Hence

$$\Sigma_{\mathrm{row}}(\beta) = O_D\left(\frac{\mathrm{N}(\alpha)^{1/4} \log(2 + \mathrm{N}(\alpha))}{\sqrt{D}}\right),$$

which is of lower order than $O_D(\mathrm{N}(\alpha)/D)$. Substituting these estimates into Theorem 5.1 yields the claimed time bound. □

Proposition 5.7. *Assume that $0 \neq \alpha \in \mathcal{O}_D$ is representable, and let $\beta = u^{2k}\alpha$ be the balanced element used by Algorithm 3, where $u \in \mathcal{O}_D$ is a totally positive unit. Suppose that, while building $S(\beta)$, one stores one root $x_s \in B(\beta)$ with $x_s^2 = s$ for each $s \in S(\beta)$, and that while building $T_2(\beta)$ one stores one witness pair (s_1, s_2) for each $t = s_1 + s_2 \in T_2(\beta)$. Then one can recover roots $y_1, \ldots, y_{\ell_D(\alpha)} \in \mathcal{O}_D$ such that*

$$\alpha = y_1^2 + \cdots + y_{\ell_D(\alpha)}^2.$$

For lengths $1, 2, 3, 4$, the extra work after constructing $S(\beta)$ and $T_2(\beta)$ is linear in $M(\beta)+|T_2(\beta)|$. For length 5, a recursive search over $s \in S(\beta)$ together with four-square reconstruction for $\beta - s$ yields a minimal decomposition with worst-case extra cost $O(M(\beta)^3)$.

Proof. Let $\gamma = u^{-k} \in \mathcal{O}_D$, so that $\alpha = \gamma^2\beta$. Any decomposition $\beta = z_1^2 + \cdots + z_m^2$ therefore rescales to $\alpha = (\gamma z_1)^2 + \cdots + (\gamma z_m)^2$.

If $\ell_D(\beta) = 1$, then $\beta \in S(\beta)$ and the stored root x_β gives the desired one-square decomposition. If $\ell_D(\beta) = 2$, then the successful test in the second loop of Algorithm 3 yields $\beta = s + (\beta - s)$ with both summands in $S(\beta)$, and the stored roots of these two squares recover

a two-square decomposition. If $\ell_D(\beta) = 3$, then for some $s \in S(\beta)$ the difference $\beta - s$ lies in $T_2(\beta)$; the stored witness pair for $\beta - s$ and the stored root for s recover three roots. If $\ell_D(\beta) = 4$, then the successful test $\beta - t \in T_2(\beta)$ provides two witness pairs in $T_2(\beta)$ and hence four roots. In all of these cases the rescaling by γ gives a minimal decomposition of α.

If $\ell_D(\beta) = 5$, then there exists some $s \in S(\beta)$ such that $\ell_D(\beta - s) = 4$. Trying all $s \in S(\beta)$ and applying the previous four-square reconstruction to each representable remainder eventually succeeds and yields five roots of β; rescaling again gives five roots of α. The worst-case extra cost is $O(M(\beta)\,|T_2(\beta)|)$, which is contained in $O(M(\beta)^3)$. □

6. Pythagoras numbers and integral Pythagoras elements

For maximal real quadratic orders, the value of the Pythagoras number is already known by Theorem 2.5. The algorithmic task is therefore to turn that classification into an explicit routine producing both the value $P(\mathcal{O}_D)$ and a witness of maximal length.

We first record convenient candidate elements. Their displayed decompositions prove that they are representable by $P(\mathcal{O}_D)$ squares or fewer; the extremality will be verified algorithmically by Algorithm 4.

Proposition 6.1. *For the following choices of $a_D \in \mathcal{O}_D$, the displayed identity is a decomposition into at most $P(\mathcal{O}_D)$ squares:*

$$\begin{aligned}
6 + 2\sqrt{2} &= 1^2 + (\sqrt{2})^2 + (1+\sqrt{2})^2,\\
9 + 4\sqrt{3} &= 1^2 + 1^2 + (2+\sqrt{3})^2,\\
\frac{7+\sqrt{5}}{2} &= 1^2 + 1^2 + \omega_5^2,\\
10 + 2\sqrt{6} &= 1^2 + 1^2 + 1^2 + (1+\sqrt{6})^2,\\
11 + 2\sqrt{7} &= 1^2 + 1^2 + 1^2 + (1+\sqrt{7})^2,\\
12 + 2\sqrt{13} &= 1^2 + 1^2 + 1^2 + \omega_{13}^2 + (1+\omega_{13})^2.
\end{aligned}$$

Moreover, for the generic 5-square cases one has

$$\begin{aligned}
7 + (1+\sqrt{D})^2 &= 2^2 + 1^2 + 1^2 + 1^2 + (1+\sqrt{D})^2 \qquad (D \equiv 2,3 \pmod 4),\\
7 + \omega_D^2 &= 2^2 + 1^2 + 1^2 + 1^2 + \omega_D^2 \qquad (D \equiv 1 \pmod 4).
\end{aligned}$$

Proof. This is a direct verification. □

Algorithm 4: Pythagoras number and integral Pythagoras element

Input : A squarefree integer $D \geqslant 2$.
Output : A pair $(P(\mathcal{O}_D), a_D)$ with $\ell_D(a_D) = P(\mathcal{O}_D)$.

```
Determine p = P(O_D) using Theorem 2.5;
if D = 2 then
    Set C ← {6 + 2√2};
else
    if D = 3 then
        Set C ← {9 + 4√3};
    else
        if D = 5 then
            Set C ← {(7 + √5)/2};
        else
            if D = 6 then
                Set C ← {10 + 2√6};
            else
                if D = 7 then
                    Set C ← {11 + 2√7};
                else
                    if D = 13 then
                        Set C ← {12 + 2√13};
                    else
                        if D ≡ 1 (mod 4) then
                            Set C ← {7 + ω_D^2};
                        else
                            Set C ← {7 + (1 + √D)^2};
foreach a ∈ C do
    if Algorithm 3 applied to (D, a) returns p then
        return (p, a);
return FAIL;
```

Theorem 6.2. *Algorithm 4 is correct.*

Proof. The value p computed in the first line is correct by Theorem 2.5. By Proposition 6.1, every candidate in the list $\mathcal{C}$ is representable by at most p squares. The classical classification of quadratic Pythagoras numbers, in the form recorded by Kraśenský, Raška and Sgallová [14, Theorem 3.1], shows that in each case the chosen candidate is in fact extremal, i.e. has length p. Algorithm 3 is correct by Theorem 4.11, so the verification step returns exactly when the candidate has length p. Therefore the algorithm outputs a correct pair $(P(\mathcal{O}_D), a_D)$, and the `FAIL` branch is unreachable. □

Remark 6.3. The maximal-order hypothesis matters. For arbitrary quadratic orders there is one further exceptional case, namely the non-maximal order $\mathbb{Z}[\sqrt{5}]$, whose Pythagoras number

equals 4 [14, Theorem 3.1]. Extending the present algorithm from $\mathcal{O}_D$ to arbitrary quadratic orders should amount to replacing the integral basis, Peters' representability criterion, and the Pythagoras classification by their order-theoretic analogues; the finite-search part of Sections 4 and 5 would remain essentially unchanged.

7. Worked example and computational data

This section gives an end-to-end execution of the algorithm and a small table of computations over $\mathcal{O}_{10} = \mathbb{Z}[\sqrt{10}]$.

Example 7.1 (An end-to-end length-5 computation in $\mathcal{O}_{10}$)**.** Let

$$\alpha = 18 + 2\sqrt{10} \in \mathcal{O}_{10}.$$

We apply the algorithms step by step.

Step 0: balancing. The Pell-unit balancing step of Algorithm 3 leaves this example unchanged: α is already balanced, so the search is carried out on $\beta = \alpha$.

Representability. Here $D = 10 \equiv 2 \pmod 4$, $a = 18$, $b = 2$, and

$$\mathrm{N}(\alpha) = 18^2 - 10 \cdot 2^2 = 284.$$

Since b is even and the nearest integer to $a/(2D) = 18/20$ is $c_0 = 1$, we get

$$(2Dc_0 - a)^2 = (20 - 18)^2 = 4 \leqslant 284.$$

Hence Algorithm 1 returns `TRUE`.

Exact row search. The implementation and the formal Algorithm 2 use only exact trace bounds and sign tests. Here

$$\mathrm{Tr}(\alpha) = 36, \qquad V_\alpha^{\mathrm{tr}} = \left\lfloor \sqrt{\frac{\mathrm{Tr}(\alpha)}{2 \cdot 10}} \right\rfloor = 1,$$

so only the rows $v = -1, 0, 1$ can occur. For $v = 0$, Proposition 4.5 gives the enclosure

$$I_0^{\mathrm{tr}}(\alpha) = [-4, 4],$$

and Proposition 4.6 refines this to the exact admissible row $J_0(\alpha) = [-3, 3]$. For $v = 1$, the trace enclosure is $I_1^{\mathrm{tr}}(\alpha) = [-2, 2]$, and the exact admissible row is $J_1(\alpha) = [0, 1]$. For $v = -1$, one again has $I_{-1}^{\mathrm{tr}}(\alpha) = [-2, 2]$, and the exact admissible row is $J_{-1}(\alpha) = [-1, 0]$. Hence

$$B(\alpha) = \{-1 - \sqrt{10}, -\sqrt{10}, -3, -2, -1, 0, 1, 2, 3, \sqrt{10}, 1 + \sqrt{10}\},$$

and squaring these elements yields $S(\alpha) = \{0, 1, 4, 9, 10, 11 + 2\sqrt{10}\}$.

Exact length. One checks immediately that $\alpha \notin S(\alpha)$. Next,

$$T_2(\alpha) = S(\alpha) + S(\alpha)$$

contains, among others,

$$0, 1, 2, 4, 5, 8, 9, 10, 11, 11 + 2\sqrt{10}, 12 + 2\sqrt{10}, 13, 14, 15 + 2\sqrt{10}, 18,$$
$$19, 20, 20 + 2\sqrt{10}, 21 + 2\sqrt{10}, 22 + 4\sqrt{10},$$

but not $18 + 2\sqrt{10}$. Therefore $\ell_{10}(\alpha) \neq 1, 2$. For the three-square test, the six differences

$$\alpha - s \qquad (s \in S(\alpha))$$

are

$$18 + 2\sqrt{10},\ 17 + 2\sqrt{10},\ 14 + 2\sqrt{10},\ 9 + 2\sqrt{10},\ 8 + 2\sqrt{10},\ 7,$$

and none belongs to $T_2(\alpha)$. Hence $\ell_{10}(\alpha) \neq 3$. For the four-square test, one checks that there is no $t \in T_2(\alpha)$ with $\alpha - t \in T_2(\alpha)$ either. By Theorem 4.9, it follows that

$$\ell_{10}(18 + 2\sqrt{10}) = 5.$$

Since $P(\mathcal{O}_{10}) = 5$ by Theorem 2.5, this is a Pythagoras element of $\mathcal{O}_{10}$.

A small computational run over $\mathcal{O}_{10}$ already illustrates the distribution of exact lengths. More precisely, we considered the finite set

$$\mathcal{E}_{10}(40) = \left\{ \begin{array}{c} a + b\sqrt{10} \in \mathcal{O}_{10} : a, b \in \mathbb{Z},\ a + b\sqrt{10} > 0,\ a - b\sqrt{10} > 0, \\ 1 \leqslant \mathrm{Tr}(a + b\sqrt{10}) = 2a \leqslant 40 \end{array} \right\},$$

which contains 134 totally positive elements. The counts were obtained from a direct implementation of Algorithms 1–3; the supplementary script `real_quadratic_sos_table.py` included with the source bundle computes the counts and minimal examples underlying Table 1.

Exact length	Count with $\mathrm{Tr}(\alpha) \leqslant 40$	Smallest example(s) in $\mathcal{O}_{10}$ at minimal trace
1	11	1
2	22	2
3	14	3
4	9	7
5	2	$18 \pm 2\sqrt{10}$
∞	76	$4 \pm \sqrt{10}$

TABLE 1. Exact-length distribution in $\mathcal{O}_{10}$ for totally positive elements of trace at most 40.

At minimal trace, the elements $1, 2, 3, 7, 18 \pm 2\sqrt{10}$ realise the finite lengths $1, 2, 3, 4, 5$, respectively, while $4 \pm \sqrt{10}$ give the first non-representable examples. This small experiment already shows why the element-wise problem is different from the family-wise questions studied in [7, 19]: even within one fixed ring and a narrow trace range, all possible finite lengths and many non-representable elements already appear.

Balancing ablation on a Pell orbit. Let $u = 19 + 6\sqrt{10}$ be the positive norm-one Pell unit of $\mathcal{O}_{10}$ used by the implementation, and set

$$\alpha_k = u^{2k}(18 + 2\sqrt{10}) \qquad (k \geqslant 0).$$

By Lemma 2.2(d), one has $\ell_{10}(\alpha_k) = 5$ for every k. Table 2 compares the raw search geometry with the balanced search geometry used by Algorithm 3. The structural columns are machine-independent. The timing columns are medians of three runs of the reference CPython implementation and are included only to illustrate the scale of the balancing effect.

k	$\mathrm{Tr}(\alpha_k)$	$V^{\mathrm{tr}}_{\mathrm{raw}}$	$\mathrm{rows}_{\mathrm{raw}}$	$\lvert B\rvert$	$\lvert S\rvert$	$\mathrm{ms}_{\mathrm{raw}}$	$V^{\mathrm{tr}}_{\mathrm{bal}}$	$\mathrm{rows}_{\mathrm{bal}}$	$\mathrm{ms}_{\mathrm{bal}}$
0	36	1	3	11	6	0.017	1	3	0.017
1	35076	41	83	11	6	1.147	1	3	0.017
2	50579556	1590	3181	11	6	67.798	1	3	0.017
3	72935684676	60388	120777	11	6	4039.467	1	3	0.017

TABLE 2. Balancing ablation on the Pell orbit of $18 + 2\sqrt{10}$ in $\mathcal{O}_{10}$. Without balancing, the trace-row bound and the number of scanned rows explode along the orbit, while the balanced search remains constant.

The exact set of relevant roots and squares does not change along the orbit: every balanced search still sees $|B| = 11$ admissible roots and $|S| = 6$ distinct squares. What grows is only the amount of wasted row scanning in the unbalanced coordinates. In particular, the balancing step is not a cosmetic optimisation; it is what keeps the search phase in the norm-sized regime predicted by Corollary 5.6. The timings in Table 2 measure only the search-set construction from Algorithm 2; the current implementation still spends $O(|k|)$ exact preprocessing steps to reach the balanced representative $u^{2k}\alpha$.

Cross-field distributions and validation. To probe the behaviour beyond $\mathcal{O}_{10}$, Table 3 records exact-length distributions for six representative fields and trace bound 80. The last column $\tau_{\max}$ denotes the smallest trace at which the maximal possible finite length $P(\mathcal{O}_D)$ first appears.

D	#1	#2	#3	#4	#5	#∞	$\tau_{\max}$
5	54	576	818	0	0	0	7
10	18	87	112	29	4	270	36
13	35	256	511	78	4	14	24
29	25	130	244	121	22	60	29
53	16	75	116	85	26	126	41
101	14	32	40	28	6	200	65

TABLE 3. Cross-field exact-length distributions for totally positive nonzero elements of trace at most 80. The column $\tau_{\max}$ records the smallest trace at which the maximal length $P(\mathcal{O}_D)$ first occurs.

The same script also regenerates Figure 1. The plot is included mainly as a software-evaluation diagnostic: it separates the deterministic structural effect of balancing from machine-dependent timing. Along this Pell orbit the balanced structural search parameter remains small while the raw trace-row bound grows rapidly.

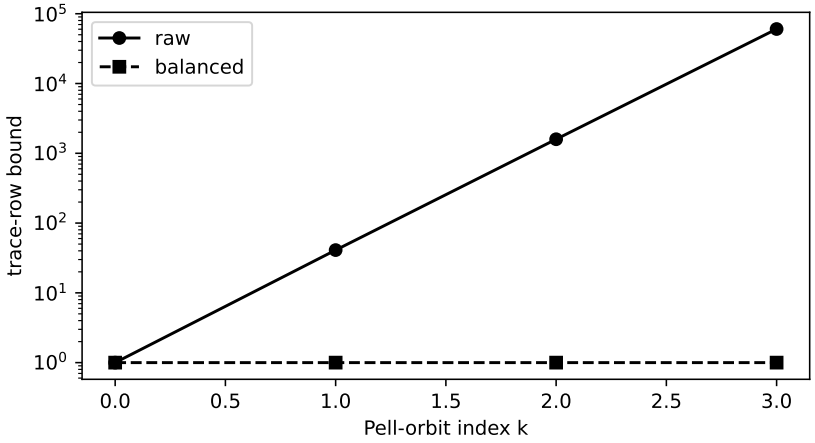

FIGURE 1. Raw and balanced trace-row bounds along the Pell orbit in the balancing ablation.

These data illustrate several phenomena already invisible in a single-field table. In the ternary case $D = 5$ every totally positive element of trace at most 80 is representable and the maximal length is already present at trace 7. By contrast, for larger D one sees substantial pools of non-representable elements, together with a delayed first appearance of maximal-length elements. Even among fields with $P(\mathcal{O}_D) = 5$, the distribution of lengths below trace 80 changes considerably with D.

Finally, we compared the package output against a separate brute-force coefficient-box validator. It scans a naive coefficient box, does *not* use the exact row-search engine, Peters' criterion, or the closed-form Pythagoras classification, and reuses only the exact coefficient arithmetic and embedding-comparison primitives from the ring layer. The sweep covered every squarefree $D \leqslant 41$ and every nonzero totally positive element of trace at most 24.

Fields checked	Trace bound	Elements compared	Mismatches
26	24	1358	0

TABLE 4. Bounded-trace validation against a separate brute-force coefficient-box implementation.

This validation sweep does not replace proof, but it provides a useful computational sanity check: it confirms that the exact-length outputs agree with a deliberately naive coefficient-box search on a substantial bounded corpus, without relying on Peters' criterion, the row-search engine, or the closed-form Pythagoras classification. The sweep concerns lengths only; constructive decompositions are checked separately in the bundled test suite.

Remark 7.2 (Implementation)**.** The companion Python package `quadratic_sos`, available at https://github.com/tomaszkania/quadratic_sos, follows the exact-arithmetic philosophy of the paper all the way through. The representability test already uses only exact coefficient arithmetic by Proposition 3.1. In the search phase, the implementation stores points as coefficient pairs in the basis $1, \omega_D$ and computes each admissible row exactly: for a fixed v, the trace inequality $\mathrm{Tr}(x^2) \leqslant \mathrm{Tr}(\alpha)$ gives a finite enclosing interval for u, and a directed

binary search based on exact sign tests for σ_1 and σ_2 then locates the precise interval of admissible integers on that row. No floating-point approximation is needed for the search. The implementation also applies unit-square balancing before enumeration, using the positive norm-one Pell unit discussed in Remark 5.5. It mirrors the paper's mathematical assumptions at the API level: the constructor rejects non-squarefree radicands D, and the exact-length and decomposition routines are defined only for nonzero inputs. The repository contains a test suite reproducing the examples and Table 1, together with an executed notebook illustrating the algorithms. The scripts `real_quadratic_sos_table.py`, `balancing_ablation.py`, `cross_field_distributions.py`, and `validation_sweep.py` compute the data underlying the computational study from this section. They are thin wrappers importing the package routines, so the published computations and the library use the same exact-arithmetic engine.

Conclusion

The package `quadratic_sos` gives a complete exact-arithmetic reference implementation for computing integral sum-of-squares lengths in maximal real quadratic orders. Peters' theorem supplies the representability criterion, exact trace-row enclosures and directed bisection enumerate the relevant squares without floating-point arithmetic, unit-square balancing controls skewed inputs, and a meet-in-the-middle step distinguishes the exact lengths $1, 2, 3, 4, 5$. The same package also computes Pythagoras numbers and explicit Pythagoras elements in the implemented class of fields.

From the software point of view, the main deliverable is a reproducible algorithm package rather than a standalone numerical benchmark. The submitted software archive contains the source package, non-interactive drivers, expected-output summaries, regression tests, scripts regenerating the tables and figures, and a notebook illustrating typical use. The implementation is deliberately pure Python: it is slower in absolute wall time than specialised compiled arithmetic in a production CAS, but it is easier to inspect, test, and adapt. Optional scripts are included for users who wish to compare against SageMath or PARI/GP installations on their own machines; they are not part of the mandatory portable test path.

Several extensions are natural. One is to combine the finite-search approach with the continued-fraction theory of indecomposable totally positive integers developed by Dress and Scharlau [4], as in Raška's work on the family-wise problem. Another is to import specialised low-rank criteria, for instance Maaß's three-square criterion over $\mathcal{O}_5$, as field-specific short-circuits. A third is to extend the algorithm to arbitrary quadratic orders, where the conductor must be tracked explicitly. Finally, the companion package `quadratic_diagonal` [8] shows how the same exact enumeration engine can be used as a building block for weighted diagonal forms.

Conflict of interest

The author declares that there is no conflict of interest.

Data availability

No external data were gathered or used in this work. All computational tables and timings were generated from the algorithms implemented in the companion repository at https://github.com/tomaszkania/quadratic_sos.

References

[1] H. Cohn and G. Pall. Sums of four squares in a quadratic ring. *Transactions of the American Mathematical Society*, 105(3):536–556, 1962.

[2] M. K. Darkey-Mensah. Algorithms for quadratic forms over global function fields of odd characteristic. *ACM Communications in Computer Algebra*, 55(3):68–72, 2022.

[3] M. K. Darkey-Mensah and B. Rothkegel. Computing the length of sum of squares and Pythagoras element in a global field. *Fundamenta Informaticae*, 184(4):297–306, 2021.

[4] A. Dress and R. Scharlau. Indecomposable totally positive numbers in real quadratic orders. *Journal of Number Theory*, 14(3):292–306, 1982.

[5] U. Fincke and M. Pohst. Improved methods for calculating vectors of short length in a lattice, including a complexity analysis. *Mathematics of Computation*, 44(170):463–471, 1985.

[6] V. Kala. Universal quadratic forms and indecomposables in number fields: A survey. *Communications in Mathematics*, 31(2):81–114, 2023.

[7] V. Kala and P. Yatsyna. Sums of squares in S-integers. *New York Journal of Mathematics*, 26:1145–1154, 2020.

[8] T. Kania. *quadratic_diagonal: Exact diagonal representability over real quadratic orders*. Software repository, 2026. https://github.com/tomaszkania/quadratic_diagonal.

[9] P. Koprowski. Solving sums of squares in global fields. In *Proceedings of the 2022 International Symposium on Symbolic and Algebraic Computation (ISSAC '22)*, pages 319–324. ACM, 2022.

[10] P. Koprowski. Algorithm 1058: Computing the group of local dyadic square-classes the easy way. *ACM Transactions on Mathematical Software*, 51(3), Article 20:1–14, 2025.

[11] P. Koprowski. Isotropic vectors over global fields. *Mathematics of Computation*, 95(359):1541–1567, 2026.

[12] T. Kowalczyk. Sums of squares of regular functions on rational surfaces. *arXiv preprint* arXiv:2407.20378, 2024.

[13] T. Kowalczyk and P. Miska. On Waring numbers of henselian rings. *Mathematika*, 70(4):e12276, 2024. doi:10.1112/mtk.12276.

[14] J. Krásenský, M. Raška and E. Sgallová. Pythagoras numbers of orders in biquadratic fields. *Expositiones Mathematicae*, 40(4):1181–1228, 2022.

[15] J. Krásenský and P. Yatsyna. On quadratic Waring's problem in totally real number fields. *Proceedings of the American Mathematical Society*, 151(4):1471–1485, 2023.

[16] H. Maaß. Über die Darstellung total positiver Zahlen des Körpers $\mathbb{Q}(\sqrt{5})$ als Summe von drei Quadraten. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 14(1):185–191, 1941.

[17] M. Peters. Quadratische Formen über Zahlringen. *Acta Arithmetica*, 24(2):157–164, 1973.

[18] M. Peters. Summen von Quadraten in Zahlringen. *Journal für die reine und angewandte Mathematik*, 268–269:318–323, 1974.

[19] M. Raška. Representing multiples of m in real quadratic fields as sums of squares. *Journal of Number Theory*, 244:24–41, 2023.

[20] R. Scharlau. On the Pythagoras number of orders in totally real number fields. *Journal für die reine und angewandte Mathematik*, 316:208–210, 1980.

[21] R. Schroeppel and A. Shamir. A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems. *SIAM Journal on Computing*, 10(3):456–464, 1981.

Institute of Mathematics, Czech Academy of Sciences,, Žitná 25, 115 67 Prague 1, Czech Republic, & Institute of Mathematics, Jagiellonian University, Łojasiewicza 6, 30-348 Kraków, Poland

Email address: tomasz.marcin.kania@gmail.com